

Ali Moallin

Electrical and Computer Engineering

For the architecture declaration of the shift register module I used a compilation of if and else if statements. Using conditions for the clock, clear, hold, shift right, shift left and parallel hold. These conditions made the 4-bit synchronous shift register work perfectly. This can all be seen in [1].

The wave form in [2] acts appropriately, with CLK toggling and CLRN going low then high. Both q and d also prove to show the correct states.

The VHDL code for the architecture declaration of the counter module started off with the priority order of Load, Enable, Hold. The load being the “if” conditional statement that ignores everything when LD = 1. The enable being the “elsif” condition for when EN = . The hold being the “else” conditional for when Q<=Q. This can be found in [3].

The timing diagram for the counter shows the correct operation, giving q[3:0] = 1111, the UD = 0, making the direction go downward. Also when the counter hits 0000, RCO spikes, which is expected. This can be seen in [4].

The VHDL code for the mealy machine [5] was basically a record of how much we have seen 01101. I made it so we update the progress on each clock edge, so once the full combination of numbers is reached, z is raised.

The wave form for the mealy machine [6] works correctly, Q gives a diamond pattern, showing multiple transitions, RCO is 1 when counting down and Q is at 0000. Each signal, ld, en, ud, q, and rco operates how it is meant to.

Appendix

```

architecture Behavioral of reg is
begin
process(CLK)
begin
    if rising_edge(CLK) then

        -- synchronous active-low clear
        if CLRN = '0' then
            Q <= "0000";

        -- hold
        elsif (S1 = '0' and S0 = '0') then
            Q <= Q;

        -- shift to the right
        elsif (S1 = '0' and S0 = '1') then
            Q <= RSI & Q(3 downto 1);

        -- shift to left
        elsif (S1 = '1' and S0 = '0') then
            Q <= Q(2 downto 0) & LSI;

        -- paralel load
        elsif (S1 = '1' and S0 = '1') then
            Q <= D;

        end if;

    end if;
end process;

```

Figure 1 (Register VHDL module code)

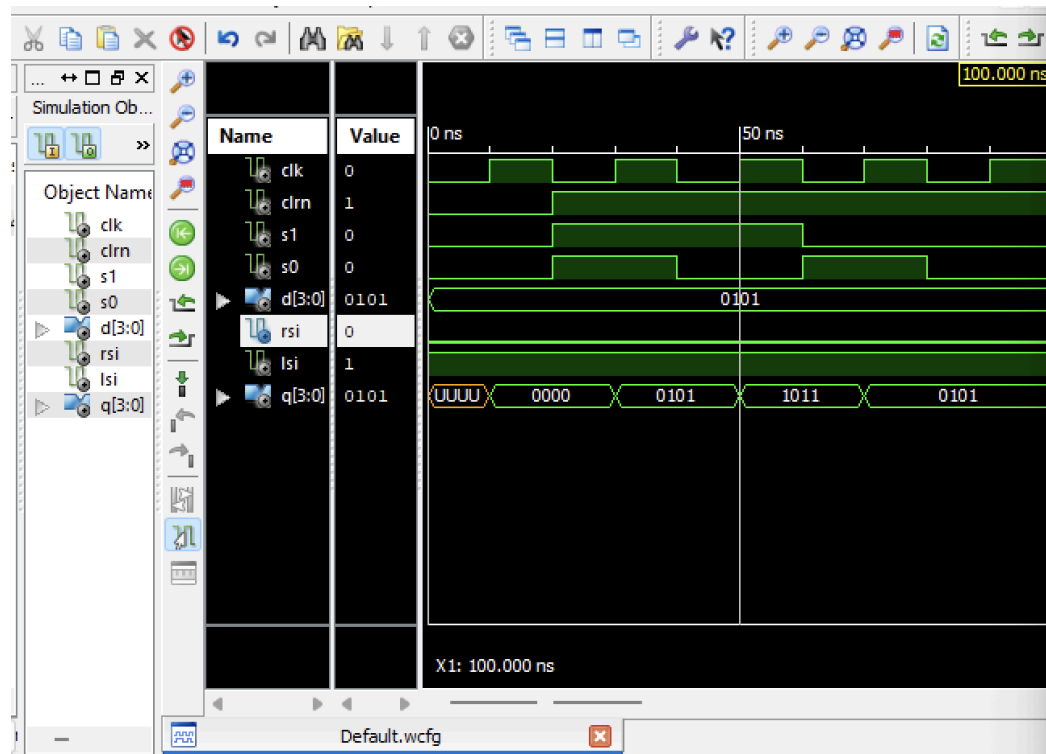


Figure 2 (Register timing diagram)

```

47
48 process(CLK)
49 begin
50     if rising_edge(CLK) then
51
52         -- LD has highest priority
53         if LD = '1' then
54             Q <= D;
55
56         -- iff not loading, check enable
57         elsif EN = '1' then
58
59             --count down
60             if UD = '0' then
61                 Q <= Q - 1;
62
63             -- count up
64             else
65                 Q <= Q + 1;
66             end if;
67
68         -- hold value if EN = 0
69         else
70             Q <= Q;
71         end if;
72     end if;
73 end process;
74
75 -- ripple-carry output logic, assert when enabled
76 -- count is at terminal value for the direction
77 RCO <= '1' when ((LD = '0') and (EN = '1') and (UD = '0') and (Q = "0000")) or
78                ((LD = '0') and (EN = '1') and (UD = '1') and (Q = "1111"))
79                else '0';
80
81 end Behavioral;

```

Figure 3 (Counter VHDL module architecture code)

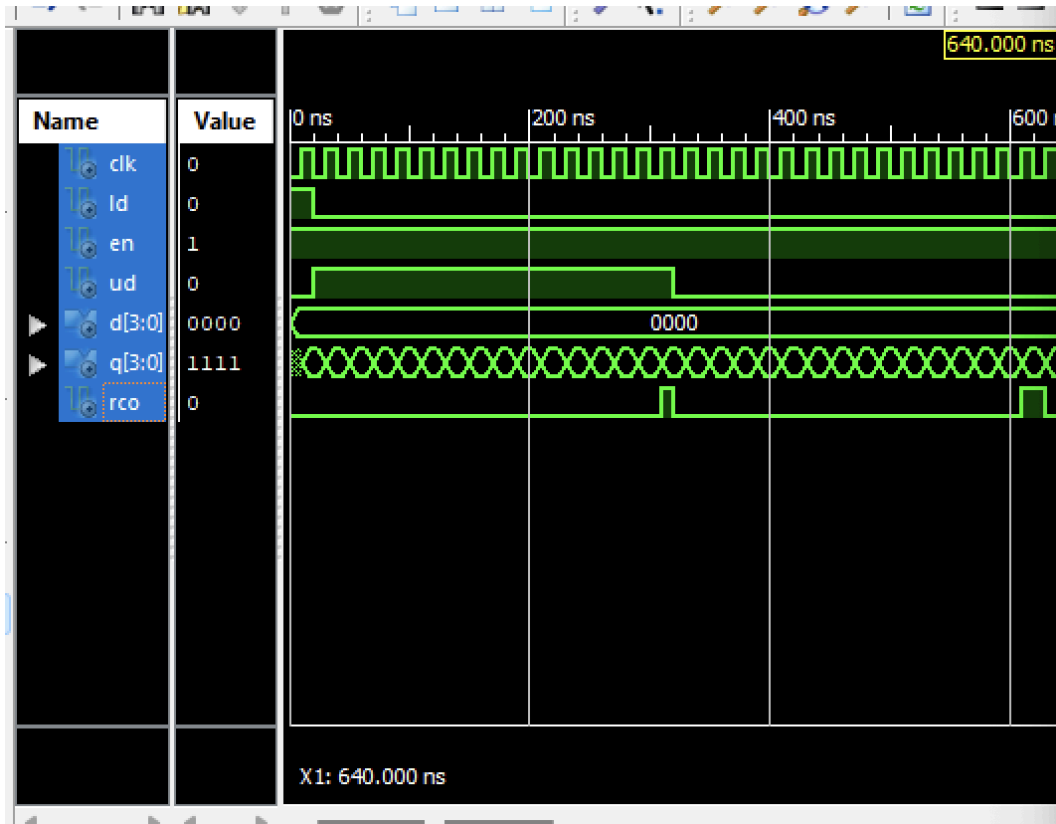


Figure 4 (Counter timing diagram)

```

38 architecture Behavioral of mealy is
39
40     type state_type is (S0, S1, S2, S3, S4);
41     signal present_state : state_type := S0;
42
43 begin
44     process (CLK)
45     begin
46         if rising_edge (CLK) then
47             case present_state is
48                 when S0 =>
49                     if X = '0' then
50                         present_state <= S1;
51                     else
52                         present_state <= S0;
53                     end if;
54                     Z <= '0';
55                 when S1 =>
56                     if X = '0' then
57                         present_state <= S1;
58                     else
59                         present_state <= S2;
60                     end if;
61                     Z <= '0';
62                 when S2 =>
63                     if X = '1' then
64                         present_state <= S3;
65                     else
66                         present state <= S1;
67

```



```

71         else
72             present_state <= S1;
73         end if;
74         Z <= '0';
75
76     when S3 =>
77         if X = '0' then
78             present_state <= S4;
79         else
80             present_state <= S0;
81         end if;
82         Z <= '0';
83
84     when S4 =>
85         if X = '1' then
86             present_state <= S2; -- overlap
87             Z <= '1';           -- DETECT HERE
88         else
89             present_state <= S1;
90             Z <= '0';
91         end if;
92
93     end case;
94 end if;
95 end process;
96
97 end Behavioral;
98
99

```

Figure 5 (Mealy VHDL module architecture)

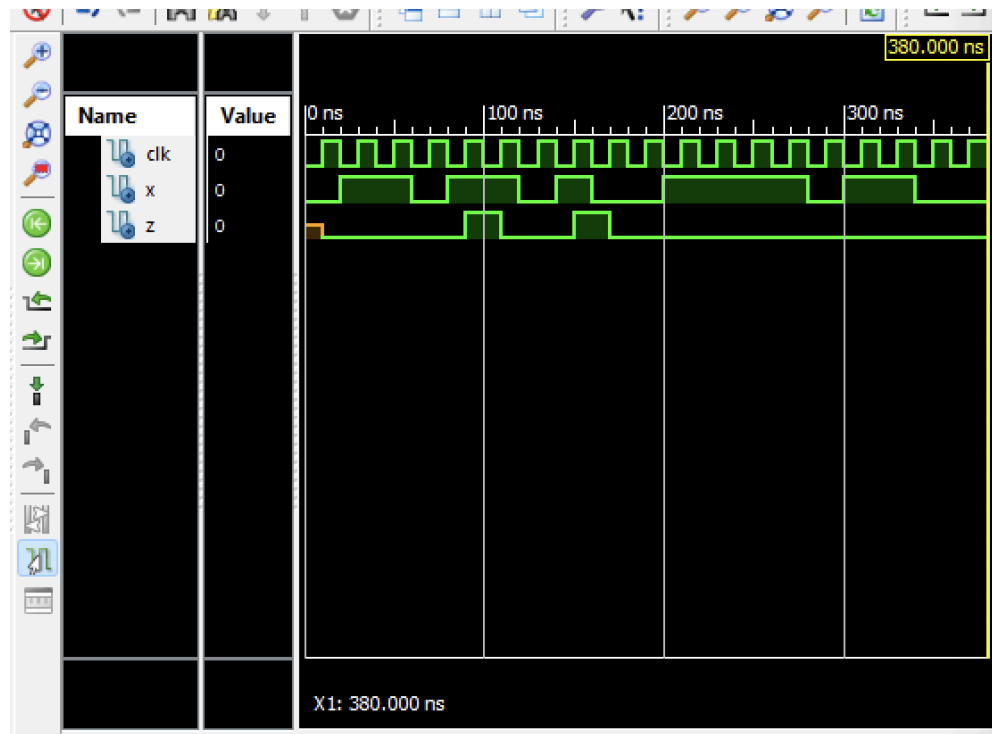


Figure 6 (Mealy timing diagram)

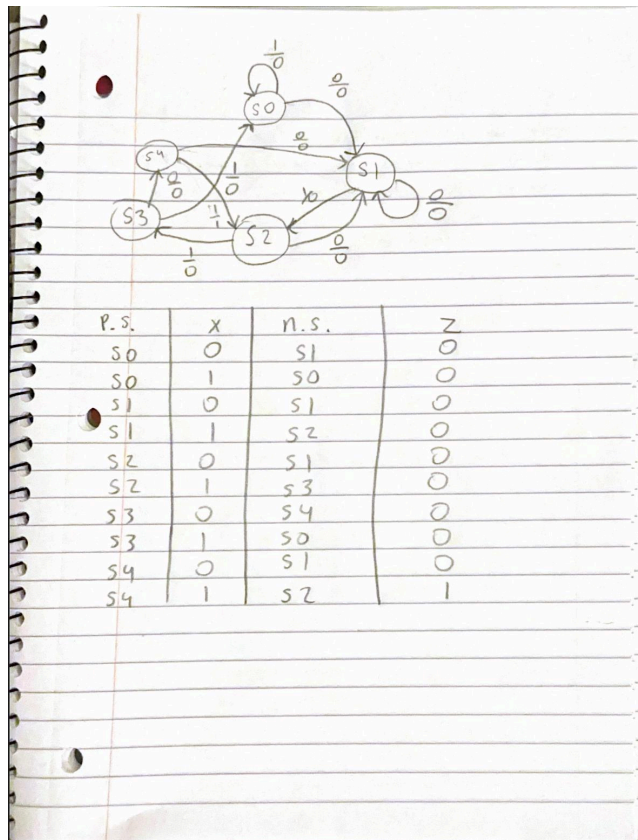


Figure 7 (Mealy state table and diagram)